FUNDAMENTOS DE INFORMÁTICA

ASESORA:
MARÍA DE JESÚS QUIJADA ORTIZ

UNIDAD 4 · ACTIVIDAD COMPLEMENTARIA 1

LÍNEA DE TIEMPO DE LOS LENGUAJES DE PROGRAMACIÓN

19 DE OCTUBRE DE 2026

GRUPO 8191 · INFORMÁTICA

Alumno: Mauro Noel Sánchez Campos

Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración

Introducción

Un lenguaje de programación es el medio con el que le damos instrucciones a una computadora. Desde los primeros programas escritos en código máquina hasta los lenguajes actuales pensados para inteligencia artificial, cada nuevo lenguaje buscó resolver un problema concreto: hacer el código más fácil de leer, más rápido, más seguro o más adecuado para un tipo de aplicación. En esta línea de tiempo presento los lenguajes más representativos con su descripción, creador, año, tipo, generación, forma de ejecución y paradigma. Para cada lenguaje incluyo la fotografía de su creador y, como segunda imagen, un pequeño programa "Hola mundo" escrito en él, para que se note cómo cambia la forma de escribir código de una época a otra.

Claves para leer la línea de tiempo

Concepto	Significado
Tipo	Bajo nivel: cercano al hardware. Alto nivel: cercano al lenguaje humano.
Generación	1.ª código máquina; 2.ª ensamblador; 3.ª alto nivel procedural u orientado a objetos; 4.ª lenguajes declarativos orientados a un fin (consultas); 5.ª basados en lógica y restricciones.
Ejecución	Compilado: se traduce todo a código máquina antes de ejecutarse. Interpretado: se traduce instrucción por instrucción al ejecutarse. Híbrido: se compila a un código intermedio (bytecode) que ejecuta una máquina virtual.
Paradigma	Imperativo/estructurado (pasos y control de flujo), orientado a objetos (clases y objetos), funcional (funciones sin estado), lógico (hechos y reglas), declarativo (qué se quiere, no cómo).

Línea de tiempo de los lenguajes de programación

1949 — Ensamblador

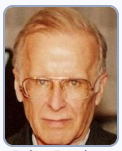

Maurice Wilkes

Usa mnemónicos (MOV, ADD) en lugar de unos y ceros; cada instrucción corresponde a una instrucción de la máquina.

Creador	Kathleen Booth (1947, primer ensamblador); EDSAC, M. Wilkes (1949)
Tipo	Bajo nivel
Generación	2.ª
Ejecución	Ensamblado
Paradigma	Imperativo

```
hola.ensa
MOV AH, 09h
MOV DX, msg
INT 21h
```

1957 — Fortran


John Backus

Primer lenguaje de alto nivel de uso amplio; pensado para cálculo científico y numérico.

Creador	John Backus (IBM)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Imperativo / procedural

```
hola.fort
PROGRAM HOLA
  PRINT *, 'Hola mundo'
END PROGRAM
```

1958 — LISP


John McCarthy

Procesa listas; base de la investigación en inteligencia artificial durante décadas.

Creador	John McCarthy (MIT)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Interpretado (también compilable)
Paradigma	Funcional

```
hola.lisp
(print "Hola mundo")
```

1959 — COBOL


Grace Hopper

Orientado a negocios; su sintaxis parecida al inglés sigue en sistemas bancarios y de gobierno.

Creador	Comité CODASYL, con influencia de Grace Hopper
Tipo	Alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Imperativo / procedural

```
hola.cobo
DISPLAY "Hola mundo".
STOP RUN.
```

1964 — BASIC

Creado para enseñar a programar; popular en las primeras computadoras personales.

Creador	John Kemeny y Thomas Kurtz (Dartmouth)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Interpretado
Paradigma	Imperativo

```
hola.basi
10 PRINT "Hola mundo"
20 END
```

1970

Pascal


Niklaus Wirth

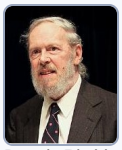
Diseñado para enseñar programación estructurada con tipos de datos estrictos.

Creador	Niklaus Wirth
Tipo	Alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Estructurado / imperativo

hola.pasc
```
begin
  writeln('Hola mundo');
end.
```

1972

C


Dennis Ritchie

Base de UNIX y de la mayoría de los sistemas operativos; da acceso directo a la memoria.

Creador	Dennis Ritchie (Bell Labs)
Tipo	Medio / alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Imperativo / estructurado

hola.c
```
#include <stdio.h>
int main(){
  printf("Hola mundo");
}
```

1972

Prolog


Alain Colmerauer

Se programan hechos y reglas, y el sistema deduce las respuestas; usado en IA y sistemas expertos.

Creador	Alain Colmerauer y Philippe Roussel
Tipo	Alto nivel
Generación	5.ª
Ejecución	Interpretado
Paradigma	Lógico

hola.prol
```
padre(juan, ana).
?- padre(juan, X).
```

1974

SQL


Donald Chamberlin

Lenguaje de consulta de bases de datos relacionales: se dice qué datos se quieren, no cómo obtenerlos.

Creador	Donald Chamberlin y Raymond Boyce (IBM)
Tipo	Alto nivel
Generación	4.ª
Ejecución	Interpretado por el gestor de BD
Paradigma	Declarativo

hola.sql
```
SELECT nombre
FROM alumnos
WHERE grupo = 8191;
```

1985

C++


Bjarne Stroustrup

Extiende C con clases y objetos; usado en videojuegos, navegadores y sistemas de alto rendimiento.

Creador	Bjarne Stroustrup (Bell Labs)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Multiparadigma (orientado a objetos)

hola.c++
```
#include <iostream>
int main(){
  std::cout << "Hola mundo";
}
```

Python

1991

Sintaxis simple y legible; domina en ciencia de datos, IA, automatización y ciberseguridad.



Guido van Rossum

Creador	Guido van Rossum
Tipo	Alto nivel
Generación	3.ª
Ejecución	Híbrido (bytecode + intérprete)
Paradigma	Multiparadigma

hola.pyth

```
print("Hola mundo")
```

JavaScript

1995

Lenguaje de la web en el navegador; con Node.js también corre en servidores.



Brendan Eich

Creador	Brendan Eich (Netscape)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Interpretado / compilado JIT
Paradigma	Multiparadigma (eventos, funcional, objetos)

hola.java

```
console.log("Hola mundo");
```

C#

2000

Lenguaje principal de la plataforma .NET; usado en aplicaciones empresariales y videojuegos (Unity).



Anders Hejlsberg

Creador	Anders Hejlsberg (Microsoft)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Híbrido (CIL en .NET)
Paradigma	Orientado a objetos

hola.c#

```
Console.WriteLine("Hola mundo");
```

Java

1995

"Se escribe una vez y corre en cualquier lugar" gracias a la máquina virtual; base de Android.



James Gosling

Creador	James Gosling (Sun Microsystems)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Híbrido (bytecode en la JVM)
Paradigma	Orientado a objetos

hola.java

```
class Hola {
 public static void main(String[] a){
  System.out.println("Hola mundo");
 }
}
```

PHP

1995

Pensado para generar páginas web dinámicas del lado del servidor.



Rasmus Lerdorf

Creador	Rasmus Lerdorf
Tipo	Alto nivel
Generación	3.ª
Ejecución	Interpretado
Paradigma	Imperativo / orientado a objetos

hola.php

```
<?php
echo "Hola mundo";
?>
```

2009

Go


Rob Pike

Simple y muy rápido; pensado para servidores y la nube (Docker y Kubernetes están hechos en Go).

Creador	R. Griesemer, R. Pike y K. Thompson (Google)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Imperativo / concurrente

hola.go
```
package main
import "fmt"
func main(){
  fmt.Println("Hola mundo")
}
```

2010 (1.0 en 2015)

Rust

Da la velocidad de C con seguridad de memoria; reduce vulnerabilidades como desbordamientos de búfer.

Creador	Graydon Hoare (Mozilla)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Multiparadigma

hola.rust
```
fn main() {
  println!("Hola mundo");
}
```

2011

Kotlin

Lenguaje preferido por Google para Android desde 2019; más conciso que Java.

Creador	JetBrains
Tipo	Alto nivel
Generación	3.ª
Ejecución	Híbrido (JVM) / compilado
Paradigma	Orientado a objetos y funcional

hola.kotl
```
fun main() {
  println("Hola mundo")
}
```

2012

TypeScript


Anders Hejlsberg

Agrega tipos estáticos a JavaScript para detectar errores antes de ejecutar.

Creador	Anders Hejlsberg (Microsoft)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Transpilado a JavaScript
Paradigma	Multiparadigma

hola.type
```
let saludo: string = "Hola mundo";
console.log(saludo);
```

2014

Swift

Sustituye a Objective-C para crear apps de iPhone, iPad y Mac.

Creador	Chris Lattner (Apple)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Multiparadigma (protocolos, objetos)

hola.swif
```
print("Hola mundo")
```

Mojo

2023

Lenguaje reciente con sintaxis de Python pensado para IA y alto rendimiento en hardware especializado.

Creador	Chris Lattner (Modular)
Tipo	Alto nivel
Generación	3.ª
Ejecución	Compilado
Paradigma	Multiparadigma

hola.mojo

```
def main():
    print("Hola mundo")
```

Conclusiones personales

Al ordenar los lenguajes en el tiempo se ve una tendencia clara: cada generación aleja al programador del hardware y lo acerca a la forma en que pensamos. En el ensamblador hay que mover datos entre registros para imprimir un texto; en Python basta una línea. Pero esa facilidad tiene un costo en rendimiento, y por eso siguen vigentes lenguajes compilados como C, y aparecen otros como Rust y Go, que intentan tener lo mejor de ambos mundos.

También me llamó la atención que los lenguajes nuevos no sustituyen a los viejos: COBOL sigue en los bancos y C en los sistemas operativos. En ciberseguridad esto es importante, porque hay que leer código en muchos lenguajes: ensamblador para analizar malware, C para entender vulnerabilidades de memoria, Python para automatizar y JavaScript para revisar ataques web. Conocer de dónde viene cada lenguaje y cómo se ejecuta ayuda a entender sus fortalezas y sus riesgos.

Bibliografía

Joyanes Aguilar, L. (2020). *Fundamentos de programación: algoritmos, estructuras de datos y objetos* (5.ª ed.). McGraw-Hill Interamericana.

Montero Montiel, G., Huitrón Hernández, M. R., Bernal Kaiser, A. M., Cervantes González, G. I., Montesano Brand, R., Rosales Mendoza, D. J. y Hernández Sánchez, E. (2026). *Fundamentos de informática* [Apunte digital, Licenciatura en Informática]. Universidad Nacional Autónoma de México, Facultad de Contaduría y Administración.

Wikimedia Commons. (s.f.). *Fotografías de los creadores de los lenguajes* [Imágenes con licencia libre]. https://commons.wikimedia.org

Sebesta, R. W. (2016). *Concepts of programming languages* (11.ª ed.). Pearson.